AI Engineering Interview Q&A Bank

High-Yield System Design & Architectural Q&A Pairs

1. Core AI Paradigms

Q: How does Retrieval Augmented Generation (RAG) improve upon traditional LLMs?

RAG enhances an AI's ability to provide accurate and context-aware responses by integrating real-time information retrieval from external data sources before generating an answer. This reduces hallucinations and ensures answers are grounded in factual, up-to-date data.

Q: What is Multimodal AI and what are its key advantages?

Multimodal AI allows systems to process and integrate various types of data simultaneously, such as text, images, audio, and video. Its main advantage is enabling more dynamic, interactive, and human-like user experiences (e.g., cross-modal search or image captioning).

Q: Describe the concept of "Agentic AI."

Agentic AI represents a shift from reactive models to systems equipped with the ability to reason, plan, and autonomously execute tasks over multiple steps, often by leveraging external tools and APIs.

2. Tools & Frameworks

Q: What is Langchain and how is it utilized in Generative AI applications?

Langchain is a framework designed to build applications powered by language models. It streamlines AI workflows by enabling structured and efficient prompt chaining, stateful workflows, and easy integration with external tools (Agents) and vector databases.

Q: In the context of RAG, what is the role of Vector Databases, and can you name a few?

Vector Databases store data as high-dimensional vectors (embeddings) to enable efficient similarity search. This is crucial for RAG, as it allows the system to quickly retrieve contextually relevant documents to append to a prompt. Examples include ChromaDB and FAISS.

Q: How do tools like Langgraph and Crew AI contribute to Agentic architectures?

Tools like Langgraph and Crew AI facilitate stateful workflows and multi-agent orchestration. They provide design patterns for multi-agent system design (like ReAct agents), allowing multiple autonomous agents to collaborate, share context, and execute complex workflows systematically.

Q: What is Gradio and why is it useful in AI development?

Gradio is an open-source Python library used to quickly create customizable, web-based user interfaces for interacting with machine learning models and APIs, enabling rapid prototyping and demonstration of AI capabilities.

3. High-Priority 2026 Production & System Design (Supplemental)

Q: How do you evaluate a RAG system, and what metrics are used?

RAG must be evaluated on both retrieval (did we find the right context?) and generation (did the LLM use the context correctly?). Frameworks like Ragas or TruLens are used. Common metrics include Context Precision, Context Recall (for retrieval), and Faithfulness / Answer Relevance (for generation). Using "LLM-as-a-judge" against a "Golden Dataset" of curated Q&A pairs is the industry standard.

Q: How do you design an agentic system to prevent "runaway tool loops" and manage costs?

You must implement strict guardrails. This includes setting hard iteration caps (e.g., maximum 5 tool calls per turn), using cheaper/faster models for routing and reasoning, and implementing human-in-the-loop checkpoints for actions that mutate state or incur high costs. Observability tools like LangSmith are critical here to trace agent execution paths.

Q: In a production RAG system, when would you use Hybrid Search over standard Semantic (Dense) Search?

Standard semantic search struggles with exact keyword matches, acronyms, or specific IDs (like "Error 404"). Hybrid search combines dense vector embeddings (for semantic meaning) with sparse keyword search (like BM25). You use it when users are likely to search for both conceptual themes and exact part numbers or names.

Q: What is "LLMOps" and why is observability crucial for Agentic AI?

LLMOps involves the deployment, monitoring, and maintenance of LLMs in production. Observability is crucial for agents because their execution paths are non-deterministic. Without tracing (seeing exactly which tools the agent called, what the prompt was, and what the external API returned), it is nearly impossible to debug why an agent hallucinated or failed a task.

4. Agentic Architecture (2026 Focus)

Q: What is an "Agentic Loop" and how does it differ from a standard LLM chatbot?

An agentic loop is an iterative execution cycle where an AI operates autonomously to achieve a verifiable goal. Unlike a standard chatbot that just answers a single prompt and stops, an agentic loop continuously cycles through Reasoning (what to do next), Acting (calling external tools), and Observing (reviewing the tool's output) until the goal is met or it hits a termination condition.

Q: What are the biggest risks of implementing an Agentic Loop in production?

The biggest risks are infinite loops (getting stuck retrying a failed tool call) and "goal drift" (forgetting the original task while chasing sub-tasks). This can lead to massive token cost spikes. To mitigate this, engineers must build strict scaffolding: setting hard limits on iteration counts, implementing timeout conditions, and using human-in-the-loop checkpoints for critical actions.

5. Real-World Data & Tooling

Q: How do you handle extracting data from complex PDFs containing tables for a RAG system?

Standard semantic chunking (splitting by character count) breaks tables. To handle this, you must use vision-language models (VLMs) to process the PDF as an image, or specialized parsers (like Unstructured.io) to extract the HTML/Markdown structure of the table, ensuring the table remains intact as a single searchable chunk.

Q: Why is "Structured Output" critical for AI Agents, and how do you enforce it?

Agents must communicate with standard APIs and databases, which require strict JSON schemas. If an LLM outputs malformed JSON, the tool call fails. You enforce structured output by prompting the LLM to reply strictly in a specific JSON schema, often leveraging libraries like Pydantic in Python, or native API features like OpenAI's Structured Outputs.

Q: How do the core data abstractions differ between LangChain and LlamaIndex?

- Documents & Metadata: Both frameworks wrap raw ingested text in a **Document object with a metadata dictionary. However, LlamaIndex Document objects also natively track explicit parent/child/sibling relationships via a relationships dictionary.**

- **Chunks vs. Nodes: LangChain calls chopped text segments "chunks" (stored as Document instances). LlamaIndex explicitly calls text chunks TextNode (or Node), treating chunks as nodes in a graph that can maintain explicit pointers to parent documents or adjacent sibling nodes.**

- **Loaders: LangChain uses individual specific loaders (e.g. TextLoader, PyPDFLoader), whereas LlamaIndex provides SimpleDirectoryReader, a unified universal directory loader that auto-detects and parses mixed file types (PDF, CSV, TXT, HTML, JSON) automatically.**

6. Model Optimization, Routing & RAG Strategies (Course 1 Core)

Q: In a system design interview, when should you recommend RAG vs. Fine-Tuning?

Use RAG for dynamic, frequently changing knowledge (facts, current data, company documents) because updating data requires zero model retraining. Use Fine-Tuning when you need to change the model's behavior, tone, style, or output syntax (e.g., teaching an LLM to speak in a specific domain dialect or output a custom code format). Rule of thumb: *RAG for knowledge; Fine-Tuning for style.*

Q: How do you optimize an AI pipeline for minimum latency and minimum API cost?

Use the "Start Big, Scale Small" model routing strategy. Prototype your prompt on the largest, smartest model (e.g., GPT-4o or Claude 3.5 Sonnet) to establish your quality ceiling. Once the output is verified, evaluate the prompt on smaller, cheaper models (e.g., Llama-3-8B or GPT-4o-mini). Route simple tasks (classification, summarization) to small models, and reserve the large model only for complex agentic reasoning.

Q: What is "Small-to-Big Retrieval" (ParentDocumentRetriever) and what problem does it solve?

It solves the trade-off between search precision and context size. Large chunks muddy vector math (ruining search precision), while tiny chunks lack enough context for the LLM to write a complete answer. Small-to-Big retrieval splits documents into tiny child chunks (for vector search) and maps them to larger parent chunks. The vector database matches the tiny chunk, but passes the large parent chunk to the LLM.

Q: What is Self-Consistency Prompting and how does it improve LLM accuracy?

Self-Consistency is an extension of Chain-of-Thought. Instead of taking the LLM's first answer, you sample N independent reasoning paths for the same prompt, and take a majority vote on the final answer. This eliminates random reasoning glitches in math, logic, and multi-step tasks.

Q: What is a Mixture of Experts (MoE) model (e.g., Mixtral 8x7B) and why is it popular in enterprise AI?

Unlike dense models that activate 100% of their parameters for every word, an MoE model consists of multiple specialized sub-networks ("experts"). A router network activates only a small subset (e.g., 2 out of 8 experts) per token. This provides the intelligence of a massive 47B parameter model at the speed and cost of a 13B active parameter model.

7. Vector Operations & Distance Metrics (Course 2, Module 1)

Q: How do you choose between Cosine Similarity, Dot Product, and Euclidean (L2) Distance for vector search in RAG?

- Cosine Similarity measures the *angle* between vectors regardless of magnitude. Use it when text chunk lengths vary significantly across your dataset.

- Dot Product measures both *direction and magnitude*. It is computationally faster than cosine similarity, and mathematically equivalent to cosine similarity if all vectors are L2-normalized (unit magnitude).

- Euclidean (L2) Distance measures geometric distance between vector endpoints. Smaller distance indicates higher similarity. Most modern vector databases (e.g., Pinecone, Chroma, Qdrant) use Cosine or L2 distance by default.

Q: What is the difference between Greedy Decoding and Sampling Decoding, and how does Temperature interact with them?

- Greedy Decoding is a strategy where the LLM *always* picks the single highest-probability token. It is 100% deterministic, locks temperature to 0, and is ideal for factual RAG, data extraction, and code generation.

- Sampling Decoding allows the LLM to randomly select from top probable tokens based on a probability distribution.

- Temperature is the hyperparameter used *only* during Sampling Decoding to control the flattening of the probability curve. Low temperature (0.1–0.3) keeps responses focused and factual, while high temperature (0.8–1.5) increases randomness and creativity.

Q: In RAG architectures, what is the difference between `stuff` and `map_reduce` document combination strategies, and which one is used in production?

- stuff takes all retrieved text chunks and concatenates them into a single string injected into the {context} parameter. It is used in 95%+ of production applications because it only makes 1 API call, resulting in fast responses (1-2s latency) and low cost. Modern large context windows (128k+ tokens) make this the standard choice.

- map_reduce runs an individual LLM call for *each* chunk, then a final LLM call to synthesize the results. It is reserved for offline batch processing (e.g., overnight 500-page book summarization) because making 10+ sequential API calls creates high latency (15-30s) that ruins real-time user experiences.

8. Production LLMOps, vLLM & Caching Strategies

Q: What is vLLM and how does PagedAttention improve self-hosted LLM serving?

vLLM is an open-source, high-throughput LLM serving framework for GPU clusters. Standard PyTorch serving wastes GPU memory because Key-Value (KV) cache memory is allocated statically and sequentially, leading to high memory fragmentation. vLLM introduces PagedAttention, which manages KV-cache memory in non-contiguous pages (similar to virtual memory in operating systems). This allows GPU memory to be shared dynamic across concurrent requests, boosting serving throughput by 2x to 4x and dramatically lowering latency.

Q: How do Prompt Caching and Semantic Caching differ in production AI architectures?

- Prompt Caching (API Level): Caches static system prompt prefixes or large static RAG context documents directly on LLM provider servers (e.g., Anthropic/OpenAI). Reusing the cached prefix cuts input token costs by 50–90% and reduces Time to First Token (TTFT).

- Semantic Caching (Application Level - e.g., Redis / GPTCache): Stores past user queries and LLM answers in a vector cache. When a new query arrives, it checks if it is semantically identical ($>95\%$ vector similarity) to a past query. If matched, it returns the cached answer instantly with 0ms LLM latency and $0 API cost.

Q: Are Document classes portable between LangChain and LlamaIndex? Can you just swap import statements?

No, swapping import statements throws an AttributeError because LangChain uses doc.page_content while LlamaIndex uses doc.text. However, both frameworks provide official 1-line converters: llama_doc.to_langchain_format() and LlamaDocument.from_langchain_format(langchain_doc).

Q: What is the primary design philosophy difference between LangChain and LlamaIndex?

- **LangChain is built on a Modular & Granular design philosophy. It provides un-opinionated building blocks (prompt templates, output parsers, discrete document loaders) that engineers manually chain together. It offers max flexibility but requires heavy boilerplate and external third-party imports.**

- **LlamaIndex is built on an Integrated & Batteries-Included design philosophy. It provides high-level abstractions out of the box (SimpleDirectoryReader, VectorStoreIndex, QueryEngine) that combine ingestion, embedding, vector storage, retrieval, and LLM querying into single-line commands.**

Q: What is the difference between Embedding text into vectors and Indexing vectors into a database?

- **Embedding is a *model operation*: it passes raw text through a neural network (e.g. text-embedding-3-small) to convert words into a high-dimensional array of numbers ([0.012, -0.043, ...]).**

- **Indexing is a *database operation*: it takes those numerical vector arrays and organizes them inside a Vector DB (ChromaDB, Pinecone) using search algorithms like HNSW (Hierarchical Navigable Small World) so you can query millions of vectors in sub-milliseconds.**